

Asmos

Complete Feature Reference

Every capability in the Asmos platform, derived by reading the shipped source: the Prisma schema, the Next.js route tree, the popup generation engine, the browser widget, and the background job definitions.

Source

~/Asmos/apps/web

Stack

Next.js 16 · React 19 · Prisma 7 · Postgres

Compiled

20 August 2026

Compiled from code only. Roadmap and planning documents in the repository were deliberately excluded. Where a feature exists as an interface but is not yet wired to a backend, it is marked **Partial** rather than described as complete.

What Asmos does

Asmos is an AI conversion-optimization platform for ecommerce stores. A merchant points it at their storefront; Asmos measures the store, designs on-site popups against that brand, ships them through a single script tag, and then runs a continuous statistical tournament that reallocates traffic toward whatever converts — generating fresh challengers from behavioural evidence as it goes.

The distinguishing claim in the code is that none of the four loops require a human: measurement, design, statistical decision-making, and regeneration all close automatically. The merchant's job is to set an offer preference and, optionally, to override.

01	Platform architecture	Stack, hosting split, data model, background workers
02	Store analysis engine	Four-source measurement, CRO scan, store profile
03	Acquisition surfaces	Free scanner, calculators, blog, pricing, SEO
04	Onboarding & installation	Four-step flow, snippet verification, live preview
05	Campaign creation	Goal, offer preference, page targeting, staged generation
06	The popup design engine	Design DNA, seeded briefs, novelty memory, guardrails
07	Rendering & typography	Three templates, webfont pairings, WCAG contrast audit
08	The browser widget	Triggers, targeting, consent, sticky assignment
09	Behavioural telemetry	Session summaries, rage clicks, field abandonment
10	Traffic allocation (bandit)	Thompson sampling, empirical priors, margin weighting
11	Knockout tournament	Sequential elimination, strikes, SRM guard, rounds
12	Diagnosis & regeneration	Eight failure patterns, axis selection, explore/exploit
13	AI insights & cross-account learning	Biweekly reviews, mined patterns, human gate
14	Rewards & coupon inventory	Five reward types, code pools, redemption caps
15	Leads, analytics & reporting	Capture, dashboards, CSV exports
16	Integrations & webhooks	Signed outbound events, API-key storage, email
17	Accounts, teams & plans	Roles, invites, tiered limits, settings
18	Administration & operations	Superadmin, logs, test harness, failure recovery
19	Appendix	API surface, data model, jobs, configuration

READING THE STATUS TAGS

SHIPPED — implemented end to end in code. **PARTIAL** — the interface exists but part of the path is stubbed, and the code says so. **INTERNAL** — staff-only, not exposed to merchants.

Platform architecture

A single Next.js 16 App Router codebase serves both the marketing site and the authenticated product, split apart at request time by Host header. Long-running AI work never runs in a request — it runs on a durable job queue.

Runtime stack

LAYER	IMPLEMENTATION
Framework	Next.js 16.2 (App Router, React Server Components), React 19.2, TypeScript 5
Styling	Tailwind CSS v4 with a CSS-variable design token layer
Database	PostgreSQL via Prisma 7 with the <code>@prisma/adapters-pg</code> driver adapter
Auth	Clerk (<code>@clerk/nextjs v7</code>), with a full mock-auth adapter for local development
Background jobs	Inngest v4 — durable, checkpointed steps with a 240 s yield ceiling under Vercel's 300 s limit
AI providers	Anthropic SDK (primary), AWS Bedrock (Claude Haiku 4.5, cross-region inference profile), Google Gemini (fallback)
Headless browser	Browserless — screenshots and <code>getComputedStyle</code> extraction
Product analytics	PostHog (EU cloud) — dashboard pageviews, widget events, optional session replay
Transactional email	Resend

Two hosts, one codebase

`src/proxy.ts` — the middleware inspects the `Host` header and enforces a hard split. Marketing routes (landing, `/analyze`, `/blog`, `/pricing`) live on the apex domain; the product (`/dashboard`, `/campaigns`, `/leads`, `/analytics`, `/rewards`, `/reports`, `/integrations`, `/settings`, `/admin`, `/superadmin`, `/onboarding`, `/tools`) lives on `app.*`. Sign-in, sign-up and invite-acceptance are pinned to the app host so Clerk's session cookie is always written and read on one origin.

Requests arriving on the wrong host are 307-redirected across. Any host that is neither production domain — bare `localhost`, Vercel preview URLs, custom domains — passes through untouched, so previews and local development are unaffected.

Background workers

JOB	TRIGGER	RESPONSIBILITY
generate-campaign	Event <code>campaign.generate</code>	Full popup generation for a new campaign; writes staged progress; attaches a redeemable reward
evaluate-knockout	Event <code>campaign.evaluate</code>	Posterior-based elimination, round advancement, generation of new challengers
sweep-stale-campaigns	Cron <code>* / 30 * * * *</code>	Marks campaigns stuck in <code>GENERATING</code> past 15 minutes as <code>FAILED</code> with a readable reason
mine-cross-account-patterns	Cron <code>0 8 * * 1</code>	Weekly mining of conclusive (axis, failure-pattern, category) combinations across all accounts
/api/cron/insights	Cron <code>0 6 1,15 * *</code>	Biweekly AI performance review per active campaign; bearer-token protected

Source: `lib/inngest/*.ts` · `vercel.json` · `app/api/cron/insights/route.ts`

WHY INNGEST RATHER THAN VERCEL CRON FOR THE SWEEP

The stale-campaign sweep needs to run every 30 minutes. Vercel's Hobby tier permits only daily cron schedules, so the job is registered on Inngest's own scheduler, which calls back into the existing `/api/inngest` route.

Campaign generation runs with `retries: 0` by design — every call site that fires the event already catches a failed dispatch and marks the campaign failed immediately. The sweep exists purely for the other failure mode: the event is accepted but the function never completes.

Multi-tenancy

`Account` is the tenant boundary. Every other model reaches it by cascade: `User`, `Invite`, `Website`, `Campaign`, `Notification` hang directly off it; `Variant`, `RewardRule`, `CouponCode`, `Lead`, `CampaignEvent`, `CampaignInsight` descend from `Campaign`. Deleting an account removes the entire tree. Dashboard queries are scoped through `getOrCreateAccount()`, never by trusting a client-supplied id.

Store analysis engine

Before Asmos designs anything it measures the store. Four independent sources are consulted in confidence order, and every field records where it came from — so a field that failed extraction reads as `null` rather than as a plausible-looking default.

The four sources

- 1 The catalogue** **SHIPPED**
Shopify exposes its whole catalogue at `/products.json` with no key; WooCommerce has the Store API; everything else usually emits JSON-LD `Product` schema for SEO. One request answers what the store sells, to whom, and at what price — product count, types, tags, vendors, and a min/median/max price band in minor units with currency.
- 2 The live DOM** **SHIPPED**
A Browserless `/function` run executes real `getComputedStyle` against the rendered page. It returns the display and body typefaces, full button styling (background, foreground, radius, text-transform, letter-spacing, weight), colours ranked by painted screen area, the global border radius, the logo, ranked product photography, the `h1` and hero text, whether a competing popup already exists and its selector, the platform, and the currency.
- 3 Screenshot pixels** **SHIPPED**
When the DOM pass is unavailable, the palette is measured from the actual pixels of the captured screenshot rather than inferred.
- 4 A vision model, for judgment only** **SHIPPED**
The model is asked only for things measurement cannot supply: brand voice, audience, value propositions, and the one signature detail that makes the store distinctive. It is not asked to name hex codes or identify typefaces.

Source: `lib/storeExtraction.ts` · `lib/storeProfile.ts` · `app/api/analyze/route.ts`

The persisted store profile

Results are written to a `StoreProfile` row, one per website, so the brand survives re-analysis, campaign edits, and every subsequent tournament round. The schema groups it into what the store sells, how it looks, how it sounds, and bookkeeping:

GROUP	FIELDS
What they sell	<code>category</code> · <code>subcategories[]</code> · <code>audience</code> · <code>priceBandMin/Median/Max</code> · <code>currency</code> · <code>productCount</code>
How it looks	<code>palette (hex + areaShare + role)</code> · <code>typeDisplay</code> · <code>typeBody</code> · <code>buttonStyle</code> · <code>borderRadius</code> · <code>logoUrl</code> · <code>productImages[]</code>
How it sounds	<code>brandVoice</code> · <code>valueProps[]</code> · <code>signatureDetail</code>
Bookkeeping	<code>platform</code> · <code>detectedPopup</code> · <code>sources (per-field provenance + confidence)</code> · <code>confirmedByUser</code> · <code>analyzedAt</code>

PROVENANCE IS A FIRST-CLASS FIELD

The `sources` JSON records, per field, which of six origins produced it (`catalogue` , `dom` , `pixels` , `model` , `html` , `default`) and with what confidence. The generator can therefore weight a measured palette differently from an inferred brand voice instead of treating both as equally true.

The CRO scan

Alongside brand extraction, `/api/analyze` scores the storefront on seven conversion elements and returns a 0–100 score with a letter grade, a named top issue, and a blunt one-line verdict.

ELEMENT CHECKED	POINTS	ELEMENT CHECKED	POINTS
Popup present	20	Exit intent	10
Social proof	20	Sticky bar	10
Email capture	15	Live chat	10
Urgency	15		

The grade scale runs from A+ (90 and above) down to F (below 50). When a popup is detected the model is instructed to name the vendor — Klaviyo, Privy, Omnisend.

Five-tier provider fallback

Analysis degrades rather than fails. The chain is: Browserless screenshot → Bedrock Claude Haiku vision → direct Anthropic API → Gemini Flash → pure HTML heuristics. The heuristic tier detects popups only by known library signatures (Klaviyo, Privy, Omnisend, Wisepops, Sumo, OptinMonster, Popupsmart, Mailmunch, Sleeknote) because the bare string `popup` appears in virtually every Shopify theme and is not a signal. Results from this tier are tagged `analysisSource: "heuristic"`.

Acquisition surfaces

The public site is a lead-generation instrument in its own right. The scanner runs the real analysis engine for anyone, with no account, and hands the result to signup.

The free CRO scanner SHIPPED

`/analyze` accepts any store URL and runs the full pipeline from Section 02. `/analyze/results` renders an animated score ring, a per-element checklist, the top issue, and the verdict.

- **A real generated popup as the teaser.** `/api/analyze/generate-popup` produces an actual AI-designed, brand-matched popup for the analysed store — not a mockup. It is rate-limited to five requests per IP per minute and cached per domain for one hour.
- **Email gate.** `/api/analyze/lead` stores the visitor as an `AnalyzeLead` with their store URL, name, industry, score and grade.
- **Continuity into signup.** When someone later signs up with the same email, `getOrCreateAccount()` finds their prior scan, marks onboarding complete, creates the website record, and immediately queues a campaign generation. They land on a dashboard with work already in progress.

Source: `app/analyze/*` · `app/api/analyze/lead/route.ts` · `lib/account.ts`

Free calculators SHIPPED

TOOL	WHAT IT COMPUTES
<code>/tools/email-capture-calculator</code>	Additional subscribers, customers, and monthly / annual revenue from closing the gap between the store's current capture rate and a benchmark, given traffic, subscriber-to-customer rate and AOV. Results are emailable via <code>/api/tools/email-capture-report</code> .
<code>/tools/traffic-calculator</code>	Traffic-scaled opportunity sizing.

Content and pricing SHIPPED

- **Blog** — a typed, in-repo post store (`lib/blog/posts.ts`) with index and slug pages, cover placeholders and a newsletter capture endpoint. Current posts cover popup conversion benchmarks, email-only versus email+SMS, and why the highest-converting popup is not always the best one.
- **Pricing** — an interactive traffic slider on a logarithmic scale from 1,000 to 1.5 M monthly visitors, mapping to six brackets (\$100 Starter through \$1,500 Scale), each with its A/B-test allowance and feature list. Annual billing applies a 20% discount; a managed-service add-on is \$200/mo on Starter and Growth and included on Scale. Above 1 M visitors the slider resolves to custom pricing.
- **SEO** — shared metadata builder, JSON-LD for organization, breadcrumb, article and FAQ schemas, plus generated `sitemap.ts` and `robots.ts`.
- **Contact** — a typed inquiry form that emails the founder via Resend.

Source: `lib/pricing.ts` · `lib/calculators.ts` · `lib/seo.ts` · `lib/blog/posts.ts`

Onboarding & installation

Four steps, tracked by a persistent progress rail. Several earlier steps (conversion goal, offer selection, audience trigger, testing strategy) still exist as routes but are explicitly no longer required — the flow was deliberately shortened.

STEP	COLLECTED
1 • Profile <code>/onboarding/business-profile</code>	Owner role (founder, marketer, developer, agency, other), conversion goal, monthly traffic band, and current email platform (Klaviyo, Mailchimp, Omnisend, none, other)
2 • Consent <code>/onboarding/consent</code>	GDPR and CCPA toggles plus custom consent banner copy, stored on the account and served to the widget
3 • Connect store <code>/onboarding/connect-store</code>	Store URL, which creates the <code>Website</code> record and triggers analysis
4 • Your popup <code>/onboarding/generate-popup</code>	Full-screen staged generation view, then <code>/onboarding/launch-confirmation</code>

Install verification SHIPPED

`POST /api/websites/verify` fetches the merchant's site and looks for the Asmos snippet. It returns one of three explicit states — `{ verified: true }`, `{ verified: false, reason: "snippet_not_detected" }`, or `{ verified: false, reason: "unreachable" }` — so the UI can distinguish "you haven't installed it" from "we couldn't reach your site". Success stamps `installVerified` and `installVerifiedAt` on the website.

Store preview SHIPPED

`/store-preview` renders the merchant's own site in a frame and injects the real widget against a chosen campaign and variant, so a popup can be inspected in situ before it ever reaches a visitor. Preview mode is passed explicitly (`? preview=true`, `&preview_variant_id=`) and the widget's event writer detects it and refuses to record — preview traffic never contaminates the statistics the bandit reads.

ROBUST SCRIPT INSTALLATION

`widget.js` does not rely on `document.currentScript`, which is `null` whenever the tag is injected asynchronously — precisely what Google Tag Manager, Shopify's script-tag API and every "load third-party scripts late" theme setting do. All storage access is wrapped, so a browser in private mode or with third-party storage blocked gets a popup without frequency capping rather than no popup at all.

Campaign creation

A campaign is created from a short structured form, not a drag-and-drop builder. The merchant states the goal and the offer; Asmos designs everything else.

What the merchant chooses

CHOICE	OPTIONS
Store URL & name	Free text; the URL triggers analysis before generation
Goal	<code>BOTH</code> (discount for an email) · <code>EMAIL</code> (list building only) · <code>DISCOUNT</code> (code with no email ask)
Offer preference	<code>ai_choice</code> · <code>percentage</code> (with a maximum discount ceiling) · <code>free_shipping</code> (with an optional claim limit) · <code>fixed_prize</code> (free-text description plus quantity)
Page targeting	<code>all</code> · <code>include</code> only listed paths · <code>exclude</code> listed paths. Patterns support wildcards such as <code>/product/*</code>

Page discovery SHIPPED

Rather than making merchants recall paths from memory, `GET /api/campaigns/scrape-pages` crawls their own site and returns real pages to pick from interactively. It is dashboard-authenticated and reuses the account session, so no separate IP rate limit is needed. If the crawl finds nothing, manual entry remains available.

Staged generation SHIPPED

Creation returns immediately with the campaign in `GENERATING`; the work happens on the queue, producing one variant named **Control** plus two AI challengers. Progress is not a spinner — `Campaign.generationStage` carries one of four codes that the UI renders with human labels:

STAGE	LABEL	SHOWN TO THE MERCHANT
QUEUED	Queued	Waiting for a generation slot to open up...
AI_THINKING	AI is thinking	Analyzing your brand, store, and past results to plan the popup...
STRUCTURING	Structure is forming	Turning the AI's plan into headline, copy, layout, and variants...
SAVING	Saving your campaign	Writing the campaign and variants to your account...

On failure the stage is deliberately left in place so the error can name where it broke — "Failed while: Structure is forming" — which is the entire point of splitting the progress up. `Campaign.lastError` carries the message, and a retry action re-queues the job.

Guaranteed redeemable reward SHIPPED

A popup must never go live promising something it cannot deliver. Generation attaches a real reward as part of the same transaction, idempotently:

- **Free shipping** → a `FREE_SHIPPING` rule, capped only if the merchant named a limit; free shipping is not inherently scarce.

- **Fixed prize** → a `GIFT` rule using the merchant's own description, defaulting to 40 redemptions because physical inventory should never silently default to unlimited.
- **Percentage or AI choice** → a `COUPON` rule, plus an auto-generated pool of up to 40 single-use codes, clamped against both the per-request cap and the account's remaining lifetime code budget.

If the model concluded a discount was not the right move and produced no coupon code, nothing fake is attached — the widget's eligibility gate handles that case instead.

Conversational campaign builder PARTIAL

`/api/campaigns/chat` implements a multi-turn conversation that proposes and refines a complete campaign draft via a constrained `update_campaign` tool call — campaign type (wheel, scratch card, form), design, form fields, targeting and weighted rewards. It has full Claude and Gemini paths plus a canned mock for key-less local testing, and never mocks in production. A manual creation route exists at `/campaigns/new/manual`.

The popup design engine

This is the centre of the product. It exists to solve one problem: an LLM given identical input returns the mode of its distribution every time. Variety is therefore produced by arithmetic, not by asking the model to be creative.

Design DNA — roughly thirty composable knobs

PopupDna is the vocabulary in which a popup can differ from another one. Every field is chosen per popup, rendered by all three templates, and safe to omit — `normalizeDna()` fills any missing key, so a row written before the DNA existed still renders.

Structure & flow	
step_flow	one_step · two_step
form_layout	stacked · inline
show_field_label	boolean + label text
close_affordance	x_corner · x_outside · text_link · both
Urgency	
timer_mode	none · countdown · static_badge
timer_style	digits · bar · pill
timer_seconds	clamped 30–3600
Aesthetic school	
art_direction	editorial · bold · glass
type_pairing	editorial · bold · geometric · grotesque · brand · system
elevation	flat · raised · floating
surface_treatment	plain · paper · glow · block · mesh

Composition	
text_align	left · center
color_usage	accent_only · tinted_surface · duo_accent · saturated
offer_display	inline · hero
image_style	photo · duotone · tinted · mono
image_treatment	none · side · background · top_band
Visual detail	
corner_radius	sharp · soft · rounded · pill
button_shape	rect · rounded · pill
button_fill	solid · outline · dark
accent_placement	button_only · headline · top_border · background_block
density	compact · regular · airy
type_scale	small · medium · large
overlay_weight	none · light · medium · heavy
entrance	fade · slide_up · scale · slide_side
theme	light · dark · brand

Alongside these, the DNA carries every word of non-headline copy that used to be a string literal inside the templates: `eyebrow`, `social_proof`, `privacy_note`, `dismiss_text`, `email_placeholder`, and the full copy for the capture, reveal and success steps. A `null` on any of the optional ones means the element is not rendered at all — not that a placeholder appears.

COMBINATORIAL SIZE

Counting structure × visual × flow and ignoring free text entirely, the module documents the space as on the order of 10^8 distinct renderable popups before a single word of copy changes.

Seeded design briefs — where variety actually comes from

`lib/designBrief.ts` pre-samples the structural design space with a seeded mulberry32 RNG and hands the model a *brief* it must design within. The locked knobs are then re-applied server-side after the model responds, so a brief the model quietly ignored does not become a popup it designed. Seeds are derived from the campaign id and round via an FNV hash, which means any generation can be replayed exactly.

- **Copy angle** is sampled from eight persuasion stances — reciprocity, loss aversion, genuine scarcity, curiosity, insider/community, plain utility, concierge, celebration.
- **Copy mood** and **CTA shape** are sampled independently.
- **Art-direction dealing** distributes distinct schools across a variant set so two arms are not drawn into the same aesthetic.
- **Fingerprint distance** (`dnaDistance`) steers new draws away from recently used structures.

Novelty memory

`fetchNoveltyMemory()` reads the last 25 popups the account has been shown and passes both their headlines and their structural fingerprints in as an explicit do-not-repeat list. The sampler is the guarantee; the prompt is the polish, catching near-duplicate copy the sampler cannot see. The system prompt states the reasoning plainly: a merchant who sees the same popup twice concludes the AI does nothing.

Explore versus exploit

MODE	WHEN	DIVERGENCE POLICY
Explore	Cold start, no meaningful traffic	Every variant gets its own brief and is <i>supposed</i> to look substantially different — different template, flow, urgency treatment, copy angle. The goal is mapping which region of the design space this store responds to.
Exploit	A leader has emerged from real data	Variants inherit the control's brief and differ on exactly one knob, so the conversion delta is attributable to a specific change rather than a wholesale redesign.

In both modes one rule holds: never ship two variants a visitor could not tell apart. Two specs differing only in `delay_seconds` or a synonym-level copy change are a wasted arm.

Content and compliance guardrails SHIPPED

Enforced in the prompt and again server-side, because the prompt alone is not a control:

- **Prohibited rewards.** Nothing illegal, regulated or inappropriate — no prescription or controlled substances, weapons, ammunition, explosives, alcohol or tobacco giveaways, adult content. A blocked-term scan replaces any offending headline or subhead with a safe fallback.
- **No borrowed IP.** No real third-party brand, franchise, character or copyrighted property that is not the store's own.
- **Discount ceiling.** `discount_percent` is a structured integer, not a number regex-scraped out of copy, and is hard-clamped to the merchant's maximum. Percentage mentions in prose are clamped to match.

- **Seasonal honesty.** The current date is injected fresh per call — never baked into the module-level system prompt, which is evaluated once at process start and would go stale. Holiday framing is only permitted near its actual window.
- **Banned phrases.** An explicit blocklist of the boilerplate the product used to emit: "Limited Time Offer", "Don't Miss Out", "Get N% Off Your First Order", "Claim My Discount", "Almost there", and guilt-trip opt-outs such as "No thanks, I'll pay full price".
- **Image whitelist.** `image_url` is discarded unless it matches the curated library exactly, and `image_treatment` is forced to `none` when it is rejected — so a hallucinated photo ID renders as no image rather than a 404 on the merchant's site.
- **HTML escaping.** Model-authored copy runs on a merchant's production site; a store name or prize description echoed back through the prompt is genuinely user-controlled input and is escaped as such.

The curated image library

A single source of truth for every stock photograph, grouped by store category, with a hard curation contract: **no image may contain legible text, numerals, percentage signs, price tags or signage.** This is a correctness requirement, not a style preference — copy is generated independently of imagery, so a "50%" burned into a photograph will eventually sit above a 10% offer. The library also names the store types it cannot serve, for which `image_url` must be `null`; the engine defaults to no image, on the stated principle that an irrelevant picture is strictly worse than none.

Provider resilience

Popup generation attempts AWS Bedrock first, then the Anthropic API directly, then Gemini, each behind a 45-second timeout, with a schema-constrained tool call in every case. (The insights layer inverts the first two: Anthropic, then Gemini.) The merchant's discount ceiling defaults to 15% when unset. Analytics are read from the PostHog Query API when a personal key and project id are configured, and fall back to aggregating the Postgres `CampaignEvent` table otherwise.

Rendering & typography

The model never writes HTML. It returns a JSON spec, and the server renders it through one of three physical templates — structure only; chrome, copy and behaviour are written once and shared.

Three templates, four layout styles

TEMPLATE_ID	FORM
split-screen	Image and copy side by side; the workhorse and the dispatch fallback
corner-toast	Compact, low-intrusion corner panel
fullscreen-takeover	Full-viewport treatment that paints its own scrim over photography

Each accepts a `layout_style` of `split-left`, `split-right`, `centered` or `minimal`. Adding a fourth template requires only a render function plus two registrations.

Typography served from Google Fonts

Six pairings — editorial, bold, geometric, grotesque, brand, system. The `brand` pairing is the one that finally consumes the typefaces extracted from the store's own DOM, and only when the family name matches something Google actually serves. Two rules hold because this runs on a merchant's production site:

- Every pairing declares a complete fallback stack, so a merchant with a restrictive `font-src` CSP gets a deliberate second choice rather than the browser's default serif.
- `display=swap`, always — a popup that renders invisible text while a font downloads converts at zero.

Because display faces vary wildly in apparent size at the same pixel value, each pairing carries its own optical scale multiplier, tracking and weight, so `type_scale: large` means the same thing visually across all six.

Contrast is decided at render time, not discovered by the bandit

THE MEASUREMENT THAT MOTIVATED THIS

An audit of 400 generated popups — 16 realistic brand colours × 25 seeded design briefs — found **34.5% of CTA labels below the WCAG AA 4.5:1 threshold** and **15% below 1.6:1**, which is effectively invisible. The code is explicit that this is not only an accessibility failure but a measurement failure: an unreadable button is an arm the bandit must spend thousands of real impressions learning to avoid, when contrast is a deterministic property available for free at render time.

`lib/color.ts` provides dependency-free colour arithmetic — hex and rgba parsing, alpha compositing over a backdrop, and a numeric equivalent of the renderer's `color-mix()` so contrast is measured against the surface that actually paints rather than the untinted base, which leaves a 5–12% error. `dnaCss.ts` corrects each pair as it builds the tokens; `contrastAudit.ts` is the backstop that re-derives the handful of pairs a visitor must read and reports any failures. The CTA label is held to AA text (4.5:1); headline and body ink to AA large (3:1). The fullscreen takeover is exempted from surface measurement because it overrides every ink token over its own scrim.

`renderPopupTemplateChecked()` lets generation reject a candidate and resample rather than persist a variant nobody can use.

Shared runtime

Each template previously carried its own near-identical copy of the step machine, timer, form handler and tracking hooks — about 150 duplicated lines apiece, which is why a hardcoded countdown survived three separate attempts to make popups vary. `lib/templates/runtime.ts` now owns all of it. It resolves the step flow from which steps actually exist rather than from the goal alone, and rotates fallback copy on a hash of the popup's own headline — because a constant fallback is exactly how "Almost there" became the product.

The browser widget

One script tag. No build step, no framework, no dependencies — a single self-executing function that fetches its configuration, decides whether it may run, picks an arm, waits for the right moment, and reports back.

Trigger types SHIPPED

TRIGGER	BEHAVIOUR
Exit intent desktop	<code>mouseleave</code> with <code>clientY <= 0</code> , listener removed on fire
Exit intent touch	A sharp upward scroll flick (velocity below -1.1 px/ms while past half a viewport down), or the page becoming hidden. Both settle for 3 s first to ignore load-time jitter. Read-only listeners; nothing traps the visitor or blocks navigation.
Scroll depth	Configurable percentage, clamped to 5–95, defaulting to 50%
Time delay	Configurable seconds; a deliberate <code>0</code> fires immediately rather than being treated as "unset"

WHY MOBILE EXIT-INTENT HAD TO BE BUILT

`mouseleave` never fires on a touch device, so an exit-intent arm was effectively desktop-only while a time-delay arm served everyone. A tournament comparing them was comparing desktop traffic against all traffic — not one trigger against another — and desktop converts differently from mobile on essentially every store. Device class is now sent on every event for the same reason: a bandit that pools phone and desktop averages away a real difference.

Page targeting

Campaign-level and copied identically onto every variant's `targeting.pages`, including variants generated in later tournament rounds — so "only show on `/product/*`" is never silently reset to "show everywhere" by a new round. Modes are `all`, `include`, or `exclude`, with wildcard patterns.

Variant assignment

- **Weighted roll** across each arm's `trafficPercent`, which the allocator guarantees sums to exactly 100.
- **Sticky, but re-randomised periodically.** Holding an assignment forever while the bandit reallocates means arms favoured early accumulate a returning-visitor cohort while arms promoted later see mostly first-timers — two populations that convert differently.
- **Placeholders excluded.** A variant with no rendered code is a `GENERATING` placeholder. Serving one used to drop the visitor through to a legacy DOM builder that rendered a generic unbranded card and counted it as a real impression against a variant with no design at all. Showing nothing is now the correct behaviour.
- **Human override wins.** When a campaign has a declared winner, that variant is served regardless of splits.

Consent SHIPPED

Where GDPR or CCPA is enabled on the account, the widget renders a consent banner with the merchant's own copy before anything else happens. Nothing — not the popup, not PostHog loading, not event recording — proceeds until consent is granted. Consent state is persisted client-side under `asmos_consent`, and captured leads carry `consentGiven` and `consentAt` on the row.

Eligibility gate

`/api/widget/config` refuses to serve a campaign whose goal implies a reward but which has nothing currently redeemable — an inactive rule, one past its redemption cap, or a coupon rule with an exhausted code pool and no shared code. A popup that cannot deliver on its promise does not appear.

Frequency capping and accessibility

Last-seen timestamps in `localStorage` gate re-display, with a legacy key still read for continuity. Keyboard handling covers Escape-to-close and focus scoping; dismissal is available by close button, opt-out link, or backdrop click according to the DNA's `close_affordance`.

AMBIGUITY IS LOGGED, NEVER GUESSED

`Website.url` carries no unique constraint yet, so the config route uses `findMany` rather than `findFirst`. On a duplicate match it picks deterministically — a verified install first, then the most recent — and writes a `SystemLog` error, because a silent DB-order-dependent pick risks serving one account's campaign on another account's site.

Behavioural telemetry

Conversion rate tells you *that* something is failing. It cannot tell you why. The telemetry layer exists to separate "the offer is weak" from "they could not find the email field" — two problems with opposite fixes that look identical in the aggregate.

Event types

TYPE	MEANING
IMPRESSION	The popup was displayed
INTERACTION	Step advance or named milestone (<code>email_field_focus</code> , <code>email_first_keystroke</code> , <code>dead_click</code> , <code>rage_click</code> , <code>session_summary</code>)
SUBMISSION	The form was completed
GIFT_CLAIMED	A reward was handed out
DISMISSED	The popup was closed, carrying <code>dismissAfterMs</code>

What every event carries

A persistent first-party `visitorId` in `localStorage` — also used as the PostHog `distinct_id`, which is what makes funnels, cohorts and replay work at all; previously every visitor of a variant shared one synthetic id. Alongside it: device class, page URL, referrer, the full UTM set, maximum scroll depth as a percentage, and time on page.

The session summary

Once per popup view the runtime flushes an `INTERACTION` event with `step: "session_summary"`, carrying the interaction-level signals it measured:

SIGNAL	WHAT IT DETECTS
<code>deadClicks</code>	Clicks on things inside the popup that are not interactive
<code>rageClicks</code>	Repeated clicking in the same area within a short window
<code>abandonedField</code>	Focused the email field, typed nothing, left
<code>ctaHoverNoClickMs</code>	Time spent hovering a CTA without pressing it
<code>timeToFirstKeystrokeMs</code>	Delay from popup open to first character typed
<code>typedChars</code> · <code>fieldFocusCount</code>	Engagement depth with the form
<code>scrolledInside</code> · <code>reachedStep</code> · <code>converted</code>	How far through the popup's own funnel the visitor got

A FALSE POSITIVE THAT WAS ALREADY FOUND AND FIXED

The dead-click detector was counting a deliberate interaction as a dead click, which inflated `dead_click_sessions` and tripped the `cant_find_the_cta` diagnosis on popups where nothing was wrong. The exclusion is now explicit in the click handler — a reminder that these signals feed automated design decisions and so are held to a higher bar than dashboard metrics.

Optional session replay **SHIPPED, OFF BY DEFAULT**

The widget can load `posthog-js` directly on the merchant's page for autocapture, native rage-click and dead-click detection, and full session replay, bootstrapped with the same `visitorId` so replay and custom events correlate to one person. It is gated behind a server-side kill switch (`POSTHOG_SESSION_RECORDING_ENABLED`) that is deliberately separate from simply having a PostHog key configured, and it never loads before consent. Failures are swallowed — tracking must never break the popup.

Dual event naming

Events are forwarded to PostHog under both a legacy `widget_*` name and the canonical `asmos_popup_shown` / `asmos_popup_dismissed` / `asmos_popup_converted` names, batched to one round trip, and enriched with campaign, variant, test axis, hypothesis and motivating metric so any chart can be sliced by what the variant was actually testing.

Traffic allocation — the bandit

Thompson sampling over Beta-Bernoulli posteriors, recomputed after every impression and every conversion. No human and no AI is involved in this loop.

How an allocation is produced

- 1 **Collect arms**
ACTIVE variants only. Eliminated arms must not be re-allocated the exploration floor, and GENERATING placeholders have nothing to render.
- 2 **Fit the prior**
Empirical-Bayes from the account's own history across every *other* campaign it has run. Below 200 pooled impressions it falls back to Beta(mean 0.03, concentration 60).
- 3 **Check the evidence floor**
Requires 12 pooled successes and 100 impressions on every arm. Below that, an even split — deviating early is guessing.
- 4 **Sample**
4,000 Monte Carlo draws. Each draw samples a rate per arm from its posterior, multiplies by that arm's value margin, and awards a win to the argmax.
- 5 **Distribute**
Win probabilities become integer percentages by largest remainder, with a guaranteed 5% floor per arm, summing to exactly 100.

Four design decisions worth naming

1 · The prior is empirical, not uniform

Popup capture rates live between roughly 1% and 8%. Beta(1,1) puts half its mass above 50%, so a single conversion at 20 impressions could hand an arm 70% of traffic. The fitted prior is deliberately just the pooled rate rather than a full method-of-moments fit — with the handful of variants a young account has, the variance estimate is noise, and a wrong concentration is worse than a fixed conservative one. Concentration grows with evidence and is capped at 200 so the prior can never outweigh an arm's own data.

2 · Allocation ranks by expected value, not conversion rate

A 15% arm beats a 10% arm on capture rate essentially always, and nothing in a rate-only objective knows the 15% arm costs half again as much margin per order. Sampling stays Beta-Bernoulli, so it remains exact; the sampled rate is multiplied by the arm's own margin before the argmax:

FACTOR	EFFECT
Discount cost	$1 - 0.6 \times (\text{discount} / 100)$, floored at 0.05. A weight of 1.0 would mean "a 20% discount must convert 20% better to break even", which is only true if margin equals price; 0.6 is a conservative default for typical ecommerce gross margin.
Attention cost	$1 - 0.25 \times \text{fastDismissRate}$. A popup a third of visitors flick away inside two seconds is buying its conversions with session quality it is also destroying. Capped so it can shade a decision but never dominate it — it is a softer signal than a known cash cost.

Fast-dismiss rate is measured as a share of *impressions*, not of dismissals — an arm nobody dismisses quickly because nobody sees it is not thereby better.

3 • Largest remainder, not a residual

Giving the last arm whatever remained after rounding the others produced, at win probabilities 0.90 / 0.10 / 0.00, a split of 82 / 14 / 4 — the final arm below the 5% floor it was supposed to be guaranteed, purely because of its position in the array. Floors are now enforced first and remaining points go to the largest fractional parts. When floors alone would exceed 100, it degrades to an even split rather than emitting negative shares.

4 • An explicit recompute clock

The throttle reads a dedicated `allocationComputedAt` column. Inferring it from `max(variants.updatedAt)` meant that once the allocation stabilised no row was written, the clock stopped advancing, and every subsequent widget event ran a full 4,000-sample Monte Carlo over every arm. The window is 30 seconds; a conversion also triggers a recompute immediately, since that is exactly the signal worth reacting to.

Sample ratio mismatch detection

A chi-square goodness-of-fit test compares observed impressions per arm against the split that was actually requested, at the conventional strict $p < 0.001$ threshold with exact critical values for df 1–9 and a Wilson–Hilferty approximation beyond. It stays silent below 500 total impressions or fewer than two live arms, since there is nothing to say yet. SRM checks are deliberately strict: a false alarm costs a look, a missed one costs the whole experiment. This is the check that catches a broken assignment path — and it is the check that would have caught the widget counting impressions for popups it then declined to show.

Source: `lib/bandit.ts`

The knockout tournament

Above the bandit sits a bracket. Arms that the posterior says will not win are eliminated; when a round narrows to one survivor, that survivor becomes the control of the next round and fresh challengers are generated against it.

Elimination rules

RULE	VALUE AND REASONING
Threshold	$P(\text{best}) < 2\%$. Deliberately strict: elimination is irreversible within a round, so keeping a mediocre arm alive one more evaluation costs a few hundred impressions, while killing a good one costs the whole hypothesis.
Strikes	Two consecutive evaluations must agree. The counter resets the moment an arm climbs back above threshold, so one unlucky window cannot end a variant.
Evidence floor	1,000 campaign impressions and 25 successes before any elimination is considered.
SRM gate	If the traffic split is not being honoured, elimination halts entirely and the mismatch is logged. Results from a campaign whose arms are not receiving their allocated traffic mean nothing.
One at a time	At most one arm per evaluation, since each elimination changes the posterior of every survivor. The last arm standing can never be eliminated.
Cadence	At most one evaluation per campaign per 30-minute window, claimed atomically before dispatch so a burst of concurrent events produces one evaluation rather than one each.

WHY THIS IS NO LONGER A Z-TEST

The previous rule eliminated on a "conclusive" significance flag combined with a rate below control. Three things were wrong with it. First, "conclusive" flagged both tails of a one-sided statistic at 5%, which is a two-sided alpha of 0.10 while the comment beside it claimed 95%. Second, the function re-tested every arm on every evaluation with no alpha spending and no correction for the number of arms; repeated peeking at a fixed-horizon test inflates the false-elimination rate far past its nominal level. Third — and worst — the baseline was the previous round's *winner*, whose measured rate is upward-biased precisely because it was selected for having the highest measured rate. Challengers were being compared against an inflated control, and the bias compounded every round.

A Thompson posterior is already a sequential object: $P(\text{best})$ is valid to read at any time, needs no correction for how often you look or how many arms there are, and is computed from the same numbers driving allocation.

Round advancement

When a round is full and one arm remains, the campaign's `tournamentRound` increments and the survivor is promoted to control of the next round. Round capacity is tiered — 3 on Free through 30 on Scale — and is required to be at least 3 on every tier, because campaign creation always produces one control plus two AI variants; a lower cap made the entire regeneration branch unreachable from round one onward.

What the merchant sees

A campaign detail page with seven tabs: **Overview** (variants, impressions, submissions, conversion rate), **Knockout Bracket** (the live round plus a collapsible history of every past round), **Analytics**, **Variants**, **Performance**, **Insights**, and **Settings**.

The Performance table lists each variant with its status, traffic allocation, visitors, conversions, conversion rate, *confidence to be best*, a trend sparkline and a last-24-hours figure, with search and CSV export. The Variants tab supports adding a variant, deleting one, and declaring or clearing a winner by hand; a declared winner overrides the bandit permanently, and the traffic column is labelled "auto" while allocation is machine-managed.

Source: `lib/inngest/evaluateKnockout.ts` · `app/(dashboard)/campaigns/[id]/*`

Statistics surfaced to merchants

`lib/stats.ts` provides a two-proportion z-test as a one-sided confidence figure, an honest two-sided p-value for anywhere a p-value is displayed, and a 95% credible interval on the absolute difference in conversion rate under a weak Jeffreys prior. The last of these is preferred in the interface on the stated grounds that a merchant asking "is this better" is better served by "between +0.4 and +2.1 points" than by a p-value — and unlike a p-value, an interval does not invite the reader to treat one threshold crossing as proof.

Diagnosis & regeneration

When a round has room, the tournament does not just eliminate — it generates challengers. What it changes is chosen from a classified diagnosis of how the current popup is failing, not from a generic "try something different".

The five test axes

Every variant declares exactly one `testAxis` — `trigger`, `friction`, `copy`, `layout`, or `visual` — which is what lets the bandit and PostHog attribute a conversion delta without ambiguity. Brand tokens (palette, typefaces, signature element) are never a test axis; they are locked across the baseline and every variant.

Axes whose results are already conclusive are struck off the list and never re-tested. When all five are conclusive, generation stops.

The eight failure patterns

PATTERN	DETECTED FROM	PRESCRIBED FIX
<code>low_offer_appeal</code>	Engagement beyond display below 10%	Trigger (wrong moment) or copy (offer isn't compelling) — the funnel array disambiguates
<code>form_friction</code>	≥30 email-field focuses with conversion below 10%	Friction axis: cut fields; question whether the two-step flow is adding friction here
<code>premature_dismissal</code>	Mean dismissal under 2 s with dismiss rate above 30%	Trigger axis: a later, gentler trigger before touching copy or layout
<code>cant_find_the_cta</code>	Dead clicks in >12% of measured sessions	Layout/visual only, never copy: change button fill, shape or accent placement so the action is unmistakable
<code>interaction_rage</code>	Rage clicks in >6% of sessions	Something reads as broken — simplify to <code>one_step</code> , remove anything that looks interactive but isn't
<code>field_abandonment</code>	Focus-then-leave in >12% of sessions	The ask is the blocker: privacy reassurance, warmer placeholder, visible field label, restate the reward beside the field
<code>cta_hesitation</code>	Long hover without click in >18% of sessions	This is a copy problem: rewrite the CTA and the value restatement around it
<code>slow_to_engage</code>	Median time to first keystroke above 12 s	Too much to read before acting: cut words, raise type scale, lower density

Classification requires 30 impressions for funnel-based patterns and 20 measured sessions for UX-signal patterns; below either, the pattern is `insufficient_data` and the engine falls back to the cold-start ranked order — trigger, then friction, then copy, then layout, then visual — rather than over-fitting to noise. Thresholds are described in the code as deliberately generous, because these are diagnostic hints handed to a model that weighs them, not automated decisions.

Plain-language motivating metrics

Each generated variant must cite the specific evidence that motivated it, written for a store owner rather than an analyst — for example: *"62% of visitors who open this popup never reach the email field — testing a one-field, no-name variant"*, or *"people who dismiss are gone in under 2s on average — testing a later trigger"*. The instruction is explicit that citing the behavioural pattern beats citing a bare percentage: that is the difference between what is happening and why.

What carries forward into a new round

THE BRAND USED TO DECAY AFTER ROUND ONE

Knockout generation previously passed `brandTokens: undefined`, so every variant after round one was built from a synthetic palette derived from a single hex, with `system-ui` type and a generic imagery style. The brand was at its strongest in round one and gone by round two — exactly the period during which the bandit decides what "works" for this store.

Brand tokens now resolve in a documented source order: the persisted `StoreProfile` (durable, survives re-analysis), then the campaign's own `generationContext` (what the merchant saw at creation), then the account colour as a last resort. Page targeting and the merchant's original offer preference are carried forward on the same principle.

Generated challengers are created as `GENERATING` placeholders at 0% traffic, filled in on success, deleted if the model returned fewer specs than requested, and recorded as a `CampaignInsight` summarising each new arm's axis, hypothesis and motivating metric. The account's AI generation counter is incremented once per call regardless of how many variants it produced.

AI insights & cross-account learning

Two slower loops sit above the tournament. One explains a single campaign to its merchant. The other looks across every account for heuristics worth generalising — and refuses to apply any of them without a human.

Campaign performance reviews SHIPPED

A cron on the 1st and 15th of each month at 06:00 UTC reviews every active campaign. The model receives per-variant impressions, interactions, submissions, gift claims, traffic share and design, and returns two to four sentences of plain English about the pattern — explicitly instructed not to restate raw numbers, because the bandit already handles reallocation and the value here is explanation.

Where the data clearly supports it, the model may call a `propose_variant` tool with a complete, concrete proposal, which the merchant can accept with one click from the Insights panel. The prompt tells it not to propose a variant just to have one — to skip it when there is too little data or the current leader is already working. A manual "Generate report now" button exists and is separately rate-limited to guard against cost runaway from retry loops or repeated clicking; the cron is unaffected by that limit.

Source: `lib/insights.ts · app/api/cron/insights/route.ts · app/(dashboard)/campaigns/[id]/InsightsPanel.tsx`

Cross-account pattern mining SHIPPED

Every Monday at 08:00 UTC, a job reads each active campaign's already-computed analytics — reusing the same significance and failure-pattern logic the per-campaign evaluator uses, rather than a parallel reimplement — and groups conclusive results by **(test axis, failure pattern, store category)**. A combination observed across **five or more independent campaigns** becomes a candidate heuristic, written in plain English:

*"Across 7 independent fashion/beauty campaigns, testing the **trigger** axis conclusively improved conversion when the underlying issue was people dismissing almost immediately after the popup appears. Candidate for a general heuristic: when a fashion/beauty popup shows this pattern, prioritize the trigger axis."*

DELIBERATELY HUMAN-GATED

Candidates are written as `DRAFT` and are **never** auto-promoted. Only rows an administrator has marked `APPROVED` are woven into the live generation system prompt. The reasoning is stated in the schema: a bad pattern resting on a false-positive read would otherwise degrade every customer's popups at once. Duplicate candidates for a combination already pending or decided are suppressed.

Review happens at `/admin/learned-patterns`, where each candidate can be approved or rejected, stamped with reviewer and timestamp. Only wins are mined at present, not losses.

Source: `lib/inngest/mineCrossAccountPatterns.ts · app/(dashboard)/admin/learned-patterns/*`

Rewards & coupon inventory

Rewards belong to a campaign and are shared across all its variants. Every code is a promise of a discount, so the liability is bounded at three separate levels.

Reward types

TYPE	BEHAVIOUR
COUPON	Requires something to actually give out — a shared code, or at least one unused pool code
DISCOUNT_PERCENT	Percentage off; needs no code, can be applied automatically at checkout
DISCOUNT_FIXED	Fixed-amount discount
FREE_SHIPPING	Shipping waived
GIFT	A non-monetary prize with a free-text description instead of an amount or code

Single-use code pools

A `RewardRule` may carry a pool of individually unique `CouponCode` rows — CSV-imported, bulk-generated, or both. When a pool exists, claiming hands out one unused code rather than the shared code. Claiming is done through a conditional `updateMany` on `usedAt: null`, so two simultaneous visitors cannot receive the same code. Codes are generated from a 32-character alphabet that excludes visually ambiguous glyphs (no l, O, 0, 1).

Three independent limits

LIMIT	FREE	STARTER	GROWTH	SCALE
Total outstanding codes per account	100	1,000	5,000	20,000
Codes per generate request	25	100	250	500
Codes per import request	100	1,000	2,500	5,000

The account-wide ceiling exists because the previous guard was a flat 1–1,000 check applied identically to every tier, with nothing at all stopping repeated calls from piling up an unbounded total — a free account could hit "generate 1,000" in a loop indefinitely.

Redemption accounting

`maxRedemptions` works uniformly across every reward type, not just coupons, so "the first 200 free-shipping claims" is expressible. `redemptionsCount` increments once per lead the reward is handed to. `Lead.rewardRuleId` records which rule a lead received — necessary to attribute redemptions for non-coupon types that leave no per-lead code row.

Selection and availability

`pickWeightedReward()` performs a weighted roll across eligible rules, for wheel and scratch-card style popups. A rule is eligible only when it is active, under its cap, and — for coupons — actually has something left to give. An inactive

rule is excluded from both selection and the campaign's popup-eligibility check, so a reward can be paused without deleting it or losing its redemption history.

The Rewards dashboard SHIPPED

- Create any reward type manually and attach it to any campaign — previously the page only ever showed rewards the AI had created as a side effect of generation.
- Edit, pause and resume, or reassign a reward to a different campaign owned by the same account.
- Free-text categories for grouping ("Welcome offers", "Holiday").
- Server-side paginated, searched and filtered code management, so an account with thousands of codes never loads the whole set into the browser.
- Bulk deletion of specific codes or a "clear all unused" sweep; deleting a used code is permitted, since the lead's claimed code text is preserved on the lead row regardless.
- Full CSV export of a pool via a separate streaming endpoint.
- Per-reward totals: codes issued, codes used, redemptions against cap.

Deleting a reward cascades to its code pool and nulls the reference on any leads that redeemed it; their claimed code text stays visible either way.

Leads, analytics & reporting

Lead capture

`POST /api/widget/leads` is a single ordered transaction: create the lead, select and claim a reward, stamp the claimed code and rule, increment the redemption counter, write a `SUBMISSION` event and a `GIFT_CLAIMED` event where applicable, then return the reward to the widget for display.

Everything slow is deferred to `after()` so the widget's acknowledgement is never delayed: the bandit recompute, the outbound webhook, the PostHog forward. Reward email delivery through Resend is best-effort — if it fails, the reward still displays in the popup. Captured fields are name, email and phone, with consent state and timestamp.

Leads page SHIPPED

Every lead across the account with its email, name, originating campaign and variant, and capture time — filterable, searchable, and exportable to CSV via `/api/leads/export` with campaign and date-range parameters.

Dashboards

SURFACE	CONTENTS
Dashboard home	Active campaign count, total impressions, total submissions, emails captured, and account-wide conversion rate; a board of the five most recent campaigns each broken down by variant with control and winner markers, traffic share and per-variant conversion rate. A dedicated empty state covers new accounts.
Analytics	Account-level metric cards, per-campaign comparison rows with colour-coded variant bars, and conversion-rate ranking.
Campaign analytics tab	Total impressions, total submissions, best conversion rate, variants running, plus a traffic-allocation donut.
Reports	Date-range and campaign filters producing two CSV exports: a performance report (<code>/api/reports/performance</code>) and a leads export, both date-tagged.

Chart components

A small in-house set — `DonutChart`, `Sparkline`, `ConfidenceBar`, `StatCard`, `DataTable`, `KnockoutBracketPreview` — rather than a charting dependency.

Notifications SHIPPED

An in-app bell backed by the `Notification` model with deep links, unread counts, mark-one-read and mark-all-read, served by `/api/notifications` and `/api/notifications/read`.

NOTIFICATION PREFERENCES

PARTIAL

The Settings → Notifications tab lists five toggles — variant winner declared, install not detected, campaign went live, weekly performance digest, and new-lead milestones — rendered from a static array with default states. They are presentation only; no persistence layer backs them in the current code.

Error handling

Every dashboard route carries its own `loading.tsx` and `error.tsx`, with a global error boundary and a custom 404 above them. Client-side errors are reported through a `logError` server action into the `SystemLog` table, where administrators can read them.

Integrations & webhooks

Outbound webhooks SHIPPED

A merchant supplies an HTTPS endpoint and their own signing secret; Asmos signs each payload with it. HTTPS is enforced at save time, and the secret is masked on read — only the last four characters are ever returned.

EVENT	PAYLOAD
lead.captured	Campaign id and name, variant id and name, the lead (email, name, phone, consent flag, ISO capture time), and the reward with label, type and coupon code — or <code>null</code>
variant.winner_declared	Campaign id and name, winning variant id and name, ISO declaration time

Signatures follow the Stripe and GitHub convention — `X-Asmos-Signature: sha256=<hex>` — alongside `X-Asmos-Event`, `X-Asmos-Timestamp` and a `User-Agent` of `Asmos-Webhook/1.0`. Delivery has a 10-second timeout, is fire-and-forget, and never throws into the caller: a slow or dead customer endpoint must not delay or break a visitor's lead acknowledgement. Failures are logged. There is no retry queue in this version, and the code says so.

Integration connections PARTIAL

The Integrations page presents five providers grouped into email, ecommerce and automation categories — **Klaviyo**, **Mailchimp**, **HubSpot**, **Shopify** and **Zapier** — each with connect and disconnect flows backed by `GET` and `PATCH /api/account/integrations`. API keys are stored per integration on the account with a connection timestamp, and connection status is reported accurately.

WHAT IS NOT YET BUILT

The route's own header comment states it plainly: this only stores and returns the key — nothing currently reads it to forward leads to these providers, and that sync job does not exist. Webhooks are the working path for getting leads out of Asmos today.

Transactional email SHIPPED

Five templated messages through Resend: the reward email to a converting visitor, the team invite, the contact-form notification, the calculator results report, and a newsletter signup notification.

Billing PARTIAL

The account model carries `planTier`, `subscriptionStatus` (trialing, active, past due, canceled) and a unique `stripeCustomerId`, and the Billing tab renders the current plan, status badge and a full plan-feature comparison. Tier limits are fully enforced throughout the product. The upgrade button is disabled and the page states that payment setup is not yet connected — no Stripe integration exists in the codebase.

Accounts, teams & plans

Roles and invitations SHIPPED

Three roles — OWNER , ADMIN , MEMBER . Invites are created with a unique token, emailed through Resend, and move through PENDING → ACCEPTED or REVOKED . The Team tab lists current members with their role badges and pending invites with a revoke action. Acceptance happens at /invite/[token] , which converts the invite into a User on the account.

Plan tiers

LIMIT	FREE	STARTER	GROWTH	SCALE
Lifetime AI generation calls	10	40	150	500
Variants per tournament round	3	8	16	30
Total outstanding coupon codes	100	1,000	5,000	20,000
Codes per generate request	25	100	250	500
Codes per import request	100	1,000	2,500	5,000

WHY THE FREE TIER IS GENEROUS ON GENERATIONS

The limits file is explicit: the core loop — generate, test, let Asmos craft new variants from real analytics — is the product's whole pitch, so gating it to near-zero on Free defeats the point. Every tier is still capped so cost stays bounded, and the counter increments once per AI call regardless of how many variants that call produces.

Account settings

Five tabs. **Account** covers business name, industry and brand colour. **Team** is described above. **Billing** shows the plan and comparison. **Notifications** lists the toggles noted in Section 15. **Autonomy** offers three levels — Manual, Assisted and Autonomous.

AUTONOMY PARTIAL

The three-level control is present and saves optimistically, but there is no autonomyLevel column on the account model and nothing in the bandit or knockout code branches on it. In practice today, live traffic reallocation always runs; the merchant's real override is declaring a winner by hand, which the allocator honours permanently.

Website management

Multiple sites per account, each with its own install-verification state, timestamp, store profile and campaigns. Verification can be re-run at any time from settings.

Privacy and consent

GDPR and CCPA are independently toggleable per account with custom banner copy, both collected during onboarding and editable afterwards. The setting is served to the widget on every config fetch, and consent is a hard precondition for the popup, for analytics loading, and for event recording.

Administration & operations

Admin console INTERNAL

`/admin` is gated by an email allowlist and enforced server-side on page load, not merely hidden in navigation. It shows platform-wide economics — total accounts, total campaigns, total AI generations consumed, and aggregate impressions and submissions — over a searchable account table.

- **Account detail** — per-account campaign list with controls to change plan tier and adjust the AI generation counter. These server actions return typed results rather than letting an exception cross the action boundary, after a class of bug where the write succeeded but the client never awaited it and a later hiccup surfaced as an uncaught browser error.
- **Learned patterns** — the review queue described in Section 13.
- **System logs** — the hundred most recent `SystemLog` rows with level, message, detail and originating account.

Superadmin control centre INTERNAL

`/superadmin` lists every campaign across every account with its owner and website, and offers three actions: delete a campaign, retry a failed generation, and trigger the insights cron immediately.

Test harness INTERNAL

A floating toolkit and `/api/testing` let a superadmin skip the real-world wait for traffic to accumulate. It can inject up to 10,000 synthetic impressions against a variant at a favourable ~20% conversion rate — comfortably past the 100-impression significance floor — and dispatch a `campaign.evaluate` event on demand to force a knockout evaluation.

Failure recovery

FAILURE MODE	HANDLING
Job dispatch throws	Every call site catches it and marks the campaign <code>FAILED</code> with a readable reason immediately
Event accepted, job never finishes	The 30-minute sweep marks it failed past a 15-minute threshold and writes a system log
AI provider unavailable	Claude → Bedrock → Gemini, each behind a 45-second timeout
PostHog query fails	Silent fallback to Postgres event aggregation
Email delivery fails	Non-fatal; the reward still displays in the popup
Webhook endpoint dead or slow	10-second timeout, logged, never thrown into the request path
Model returns fewer variants than asked	Surplus placeholders are deleted rather than left as empty arms
Model returns an unreadable colour pair	Corrected at token-build time; audited again at render; generation may reject and resample
Race on account creation	Unique-constraint violation is caught and the existing row re-read
Two visitors claim the same code	Conditional update on <code>usedAt: null</code> ; only one claim succeeds
Concurrent knockout triggers	The evaluation window is claimed atomically before dispatch

Developer affordances

A complete mock-auth adapter allows running the entire product with no Clerk keys; AI generation, insights and the chat builder each fall back to canned responses when no provider key is configured, and never mock in production. A dev-only seed route populates an account with realistic campaigns, variants, events and leads. Maintenance scripts exist for backfilling website assignment and repairing blank popups. The baseline Prisma migration is committed, and the README is explicit that `db push` must not be used on tracked environments.

Appendix

A • Data model

MODEL	ROLE
Account	Tenant boundary; brand, onboarding profile, consent settings, webhook config, integration credentials, plan and subscription, AI usage counter
User	Clerk-linked member with a role
Invite	Tokenised pending teammate invitation
Website	An installed site with verification state
StoreProfile	Measured brand and catalogue understanding with per-field provenance
Campaign	Type, status, tournament round, generation context and stage, last error, declared winner, bandit and evaluation clocks
Variant	One arm: traffic share, control flag, status, round, elimination strikes, test axis, popup spec, generated HTML, hypothesis, motivating metric
RewardRule	Label, category, type, weight, shared code, redemption cap and count, active flag
CouponCode	A single-use code with claim state and claiming lead
Lead	Captured contact with consent, claimed code and reward reference
CampaignEvent	Typed event with visitor id and flexible behavioural detail JSON
CampaignInsight	An AI review with an optional structured variant proposal
LearnedPattern	A mined cross-account heuristic awaiting human review
Notification	In-app bell item with optional deep link and read state
AnalyzeLead	Pre-signup lead from the public scanner
RateLimit • SystemLog	IP throttling and platform error logging

B · API surface

Public / widget CORS

GET /api/widget/config

POST /api/widget/events

POST /api/widget/leads

POST · GET /api/analyze

POST /api/analyze/generate-popup

POST /api/analyze/lead

POST /api/contact

POST /api/newsletter

POST /api/tools/email-capture-report

Campaigns

GET · POST /api/campaigns

GET · PATCH /api/campaigns/[id]

POST /api/campaigns/[id]/variants

PATCH · DELETE .../variants/[variantId]

GET · POST /api/campaigns/[id]/insights

POST /api/campaigns/chat

GET /api/campaigns/scrape-pages

Account & team

GET · PATCH /api/account

GET · PATCH /api/account/webhook

GET · PATCH /api/account/integrations

GET · POST /api/invites

DELETE /api/invites/[id]

POST /api/invites/accept

POST /api/onboarding/business-profile

POST /api/onboarding/consent

POST /api/onboarding/website

POST /api/websites/verify

POST /api/websites/[id]/verify

Rewards, data & ops

POST /api/rewards

PATCH · DELETE /api/rewards/[id]

GET · POST · DELETE .../[id]/codes

GET .../[id]/codes/export

GET /api/leads/export

GET /api/reports/performance

GET /api/notifications

POST /api/notifications/read

GET /api/cron/insights

POST /api/testing

GET·POST·PUT /api/inngest

C · Configuration

GROUP	KEYS AND DEGRADATION BEHAVIOUR
Required	<code>DATABASE_URL</code> , plus Clerk keys or <code>MOCK_AUTH=true</code>
AI	<code>ANTHROPIC_API_KEY</code> (preferred), <code>GEMINI_API_KEY</code> (fallback), AWS credentials for Bedrock. Without any: mock in development, error in production
Analysis	<code>BROWSERLESS_TOKEN</code> . Without it, screenshots and DOM extraction are skipped and analysis falls to HTML heuristics
Analytics	<code>NEXT_PUBLIC_POSTHOG_KEY</code> / <code>_HOST</code> ; <code>POSTHOG_PERSONAL_API_KEY</code> + <code>POSTHOG_PROJECT_ID</code> for the Query API; <code>POSTHOG_SESSION_RECORDING_ENABLED</code> as a separate opt-in. All PostHog calls are no-ops when unset
Email	<code>RESEND_API_KEY</code> . Optional — lead capture works without it and delivery silently skips
Jobs	<code>INNGEST_EVENT_KEY</code> , <code>INNGEST_SIGNING_KEY</code> . Local development uses the Inngest dev CLI with no keys
Cron	<code>CRON_SECRET</code> , sent as a bearer token by Vercel Cron
Hosting	<code>NEXT_PUBLIC_APP_HOST</code> , <code>NEXT_PUBLIC_MARKETING_HOST</code> , <code>NEXT_PUBLIC_APP_URL</code>

D · Feature status summary

CAPABILITY	STATUS
Store measurement, CRO scan, store profile	SHIPPED
AI popup generation with design DNA and seeded briefs	SHIPPED
Three templates, Google Fonts pairings, WCAG contrast enforcement	SHIPPED
Widget delivery, four triggers, page targeting, consent	SHIPPED
Behavioural telemetry and eight-pattern diagnosis	SHIPPED
Thompson-sampling allocation with margin weighting and SRM	SHIPPED
Knockout tournament with sequential elimination	SHIPPED
Biweekly AI insights and one-click variant proposals	SHIPPED
Cross-account pattern mining with human approval gate	SHIPPED
Five reward types, code pools, tiered limits, redemption caps	SHIPPED
Leads, analytics, CSV reporting, notifications	SHIPPED
Signed outbound webhooks	SHIPPED
Teams, roles, invitations, tiered plan enforcement	SHIPPED
Public scanner, calculators, blog, pricing, SEO	SHIPPED
Admin, superadmin, system logs, test harness	INTERNAL
Integration lead forwarding (Klaviyo, Mailchimp, HubSpot, Shopify, Zapier)	KEY STORAGE ONLY
Stripe billing and self-serve upgrade	NOT CONNECTED
Autonomy level enforcement	UI ONLY
Notification preference persistence	UI ONLY
Conversational campaign builder	BUILT, SECONDARY PATH
Webhook retry queue	NOT BUILT

Compiled 20 August 2026 by reading `apps/web` at its current state on disk — `prisma/schema.prisma`, the full `src/app` route tree, `src/lib`, `src/components`, and `public/widget.js`. Roadmap, planning and design-intent documents in the repository were excluded by request. Every capability described above resolves to a specific file and symbol; anything the code marks as unimplemented is labelled rather than described as working.